# Project 2 – Pennine Pizza Works

## Introduction

Pennine Pizza Works is a small independent pizza takeaway. The shop currently takes orders by phone and at the counter. Staff write the order on paper and use a calculator to work out the final price. This causes problems when customers order several pizzas, choose different sizes, add toppings, request delivery or qualify for a discount. The owner has noticed that staff sometimes make mistakes when adding the order total, applying the discount or adding the delivery charge.

We were tasked with creating a program to ease the process of calculating customer order costs accurately. We will design, build, and test this program in the Flowgorithm software.

## Analysis

The business asks that the program must allow the staff to enter:

- Customer name, address and phone number
- The number of pizzas ordered
- The size of each pizza
- The number of extra toppings for each pizza
- Whether delivery is required

The program should then produce:

- Customer details
- Cost of each pizza
- Standard pizza cost and extra topping cost
- Order subtotal before discount and delivery
- Any discount applied
- Any delivery charge
- Final total cost of the order

We will make an IPO (Input, Process, Output) table to visualise these requirements. The table will go down in order from start to end:

IPO Table

| Input | Process | Output |
|---|---|---|
| Customer name, address and phone number | Displays customer name on the final order receipt. Address used if the user requests delivery. Phone number used if they need to contact the customer | Customer details |
| The number of pizzas + the size of each pizza + the number of extra toppings for each pizza | Calculates the cost for each pizza depending on size and how many toppings is on each | Cost of each pizza, including size and extra topping costs |
| N/A | N/A | Order subtotal before discount and delivery |
| N/A | If order subtotal is over £20 before delivery, apply a 10% discount. | Any discount applied |
| Whether delivery is required | Adds £2.50 to the order cost if delivery is requested | Any delivery charge |
| N/A | Calculates total cost of the order, including any discounts applied or delivery charges. | Final total cost of the order |

# Design

We will create a table in order of data type that will include all the variable functions/subroutines in the Flowgorithm code. This will allow us to understand the exact data type and purpose for each variable.

| Variables | Data Type | Purpose |
|---|---|---|
| name | String | Stores the customer's name |
| address | String | Stores the customer's address |
| delivery | String | Stores the answer given when the customer is asked if they want delivery |
| pizzasize | String | Stores the size of each pizza |

| | | |
|---|---|---|
| numberofpizzas | Integer | Stores the number of pizzas being ordered |
| numberoftoppings | Integer | Stores the number of extra toppings |
| phonenumber | Integer | Stores the customer's phone number |
| i | Integer | Used in the for loop, if the customer wants more than one pizza then the whole loop repeats (size of pizza, toppings) |
| baseprice | Real | Stores the cost of the pizza alone |
| toppingprice | Real | Stores the cost for adding extra toppings |
| itemtotal | Real | Stores the cost of the pizza + the cost of extra toppings |
| subtotal | Real | Total cost of all pizzas including toppings |
| discount | Real | If the subtotal is more than £20, this variable reduces it by 10% |
| deliverycost | Real | If the customer wants delivery, an extra cost of £2.50 is added |
| finaltotal | Real | Stores the final cost of the full order, including any discounts, delivery, number of pizzas, extra toppings, etc. This is what the customer is expected to pay |

The program flow will first start by declaring each variable's data type (string, integer, real) before asking for the customer's input. The customer is then prompted to enter their details, then their pizza order including size and toppings, then calculations happen to know if the order is eligible for a 10% discount and if there will be any additional delivery costs, then a final descriptive receipt will be output for the customer.

## Build

We will create a mock flow, from start to finish, for this program first. This is so we can understand fully how the program works before actually creating it in the Flowgorithm software.

1. Ask the customer for their name, address, and phone number.
    - Name will be used on the receipt
    - Address in case the customer requests delivery
    - Number in case the business needs to contact the customer.
2. Ask them how many pizzas they want, the size of each pizza, and if they would like any extra toppings for an extra cost.
    - Minimum pizzas 1, maximum pizzas 6
    - Small pizza - £3.25
    - Medium Pizza - £5.50
    - Large pizza - £7.15

- 1 extra topping - £0.75
- 2 extra toppings - £1.35
- 3 extra toppings - £2.00
- 4 or more extra toppings - £2.50
- If the final cost for all the pizzas combines to be over £20, apply a 10% discount.

3. Ask the user if they would like the order to be delivered.
   - Add a £2.50 delivery charge if yes
4. The program outputs the final order receipt.
   - customer's name
   - If any discount has been applied
   - If the order will be delivered or not
   - total cost of the order (including any discounts and delivery charges)

Now that we have done sufficient planning for the program, we can now create it in the Flowgorithm software.

# Test

We will test the Flowgorithm program with normal, abnormal, and extreme data before sending it to the business, ensuring it works 100% correctly without error and follows every requirement they have asked for.

(show normal, erroneous, boundary test screenshots)

Normal Testing – testing normal inputs

Fixed – The customer was able to enter letters when asked to enter their phone number. We changed the phone number value from string to integer so they could only put numbers.

Select Pizza Size (S = Small [£3.25], M = Medium [£5.50], L = Large [£7.15]):

S

How many toppings would you like? (0 or more):

1

Pizza #1 Total: £4 (Base: £3.25, Toppings: £0.75)

Would you like this order to be delivered? (Y/N):

Y

PENNINE PIZZA WORKS

ITEMISED RECEIPT

Customer Name: John

Customer Address: 123 Alphabet

Phone Number: 123456789

Number of Pizzas: 1

Order Subtotal: £4

Discount (10%): -£0

Delivery Fee: £2.5

FINAL TOTAL DUE: £6.5

- 

## Boundary Testing – testing the limits

How many pizzas would you like? (1 to 6):

1

Pizza #1

- Select Pizza Size (S = Small [£3.25], M = Medium [£5.50], L = Large [£7.15]):

How many pizzas would you like? (1 to 6):

6

Pizza #1

Select Pizza Size (S = Small [£3.25], M = Medium [£5.50], L = Large [£7.15]):

- 

Erroneous Testing – testing outside of the limits

Since the company stated that a customer can only order between 1 to 6 pizzas, any input with a number less than 1 or more than 6 should output an error message. The customer is then prompted to re-enter a value.

How many pizzas would you like? (1 to 6):

-5

Invalid. Pennine Pizza allows orders between 1 and 6 pizzas.

How many pizzas would you like? (1 to 6):

- 

How many pizzas would you like? (1 to 6):

100

Invalid. Pennine Pizza allows orders between 1 and 6 pizzas.

How many pizzas would you like? (1 to 6):

Pizza #1

Select Pizza Size (S = Small [£3.25], M = Medium [£5.50], L = Large [£7.15]):

123

Select Pizza Size (S = Small [£3.25], M = Medium [£5.50], L = Large [£7.15]):

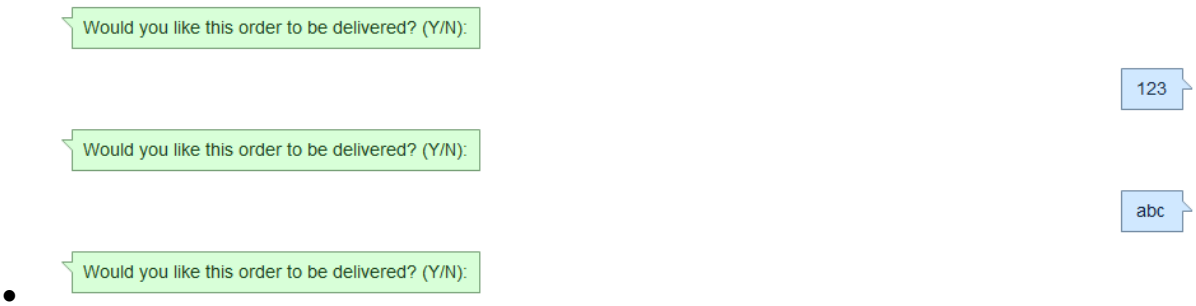
abc

Select Pizza Size (S = Small [£3.25], M = Medium [£5.50], L = Large [£7.15]):

p

Select Pizza Size (S = Small [£3.25], M = Medium [£5.50], L = Large [£7.15]):

- 




## Deploy

The full and completed Flowgorithm program has been added to our GitHub.

## Conclusion

To summarise, we have created a Flowgorithm software that allows Pennine Pizza's staff to calculate orders accurately. This includes asking how many pizzas the customer wants, calculating the costs of each pizza depending on the size and number of extra toppings, and if they want it delivered. Discount of 10% will automatically be applied if the order is above £20 (before delivery/discount costs).

The final output should be a receipt stating the customer’s details, any discounts applied or delivery information, and the total final cost of the order.